REVIEW 3 - FINAL PROJECT REPORT

AN EXPLAINABLE, SELF-HEALING DEEP LEARNING FRAMEWORK FOR ENTERPRISE NETWORK FAULT DIAGNOSIS, PREDICTION AND RECOVERY

(NetSage)

Submitted by

Jifano Oliver J P (24BCE2742)

Hrithikesh Vineeth Nair (24BAI0266)

T. Nitin Yajatkumar (24BCE0571)

Course

Computer Networks

Under the guidance of

Arivoli A

School of Computer Science and Engineering

Vellore Institute of Technology, Vellore

October 2026

TABLE OF CONTENTS

Abstract

1. Introduction

    1.1 Background and motivation · 1.2 Problem statement · 1.3 Objectives · 1.4 Scope

2. Background and Related Work

3. Requirements

4. System Architecture

5. Module Design

    5.1 Telemetry · 5.2 Dataset · 5.3 Deep learning models · 5.4 Explainability · 5.5 Prediction

    5.6 Digital twin · 5.7 Recovery playbooks and safety · 5.8 Self-healing loop · 5.9 Fault injection

6. Implementation

7. Results and Discussion

8. Testing

9. User Manual

10. Limitations

11. Conclusion and Future Work

References

List of Tables

Table 1 - Software and hardware requirements
Table 2 - Telemetry features
Table 3 - Fault classes
Table 4 - Neural network configuration
Table 5 - Simulated faults and recovery decisions of the digital twin
Table 6 - Live faults and recovery actions
Table 7 - REST API
Table 8 - Model performance
Table 9 - Per-class recall of the diagnoser
Table 10 - End-to-end self-healing results
Table 11 - Test cases

List of Figures

Figure 1 - System architecture
Figure 2 - Self-healing control loop
Figure 3 - Overview dashboard after a predicted congestion was prevented
Figure 4 - Digital twin topology and safe recovery planning
Figure 5 - Explainability view
Figure 6 - Model performance view with confusion matrix
Figure 7 - Self-healing steps during an ISP outage

ABSTRACT

Enterprise networks suffer frequent faults such as link errors, congestion, provider outages, DNS failures and wireless interference. Conventional network management relies on fixed thresholds and human operators, which makes diagnosis slow, gives no early warning, and risks making the situation worse when a hurried fix has side effects. This project presents **NetSage**, an explainable, self-healing deep learning framework that closes the loop from detection to recovery.

NetSage collects twelve telemetry features every second. A deep neural network (12-64-64-32-8) classifies the network state into eight classes (healthy and seven fault types) with 99.56% accuracy on a held-out test set, and a second network predicts whether a fault will begin within the next fifteen samples (precision 99.25%, recall 86.97%). Every decision is explained using integrated gradients, plain-language evidence and per-action reasons. Before any recovery action is applied, it is executed on a continuously synchronised **digital twin** of the network and checked against seven safety constraints covering reachability, service level, link and device load, wireless capacity, redundancy and disruption. Only actions that the twin predicts to be both effective and safe are executed; each action is then verified on the real system and rolled back if it fails.

The framework runs in two modes: a simulated enterprise campus (15 devices, 3 access points, 260 users) with seven injectable faults, and a live mode that measures the host computer's real Wi-Fi connection and injects real, self-limited faults on that computer. In end-to-end tests all seven simulated faults were diagnosed and repaired, gradual faults were frequently prevented before users were affected, and no false alarms occurred on a healthy network.

Keywords: network fault diagnosis, deep learning, explainable AI, digital twin, self-healing networks, fault prediction, AIOps.

1. INTRODUCTION

1.1 Background and motivation

Modern organisations depend on their networks for every business process. A single failed link, overloaded switch or misbehaving DNS server can stop hundreds of users. Network operations centres typically use monitoring tools that raise alarms when a metric crosses a fixed threshold. An engineer must then interpret several alarms, locate the root cause and choose a fix. This manual process is slow, depends on individual expertise, and is error-prone: a reasonable-looking action such as failing over to a backup link can overload that link and cause a second outage.

Machine learning can recognise fault patterns that are hard to capture in fixed rules, but two obstacles prevent its use for automatic repair. First, deep models are opaque, so operators cannot trust their decisions. Second, letting a model act directly on a production network is unsafe without a way to check the consequences beforehand. NetSage addresses both obstacles with explainable AI and a digital twin.

1.2 Problem statement

To design and implement a framework that automatically **diagnoses** enterprise network faults, **predicts** faults before they affect users, **explains** each decision in a form an operator can verify, and **recovers** from faults without human intervention while guaranteeing that no recovery action is applied unless it has first been shown to be safe on a digital twin of the network.

1.3 Objectives

1. Define a compact telemetry feature set that applies to both a simulated enterprise network and a real Wi-Fi connection.
2. Train a deep neural network to classify the network state into healthy and seven fault classes with high accuracy.
3. Train a second deep network to estimate the risk of a fault in the near future from recent trends.
4. Explain every diagnosis with feature attributions and plain-language evidence.
5. Build a digital twin that stays synchronised with the network and evaluates candidate recovery actions against explicit safety constraints.
6. Implement a closed self-healing loop with confirmation, verification, rollback and escalation, with optional human approval.
7. Provide a fault-injection laboratory (simulated and live) and a dashboard for demonstration and evaluation.

1.4 Scope

The project covers fault diagnosis, prediction, explanation and twin-verified recovery for a modelled enterprise campus, and monitoring plus self-scoped recovery for the host computer's live Wi-Fi link on Windows. Integration with real enterprise routers and switches (for example through SNMP or NETCONF) is outside the scope of this prototype and is discussed as future work.

2. BACKGROUND AND RELATED WORK

2.1 Traditional network fault management

Classical fault management collects device counters (for example via SNMP) and raises alarms on static thresholds. It is simple and transparent but produces alarm floods, cannot distinguish faults with overlapping symptoms (for example packet loss caused by a faulty link versus by radio interference) and provides no early warning.

2.2 Machine learning for fault diagnosis and AIOps

Machine learning approaches learn the relationship between telemetry patterns and fault types and can generalise across combinations of symptoms. Deep neural networks trained with modern optimisers such as Adam [1] and He initialisation [2] are a standard choice for tabular telemetry classification. The main criticism of these models in operations is their lack of transparency, and most such systems stop at diagnosis and leave the repair to a human.

2.3 Explainable AI

Integrated gradients [3] attribute a model's prediction to its input features by integrating the gradient of the output along a straight path from a baseline input to the actual input. The attributions satisfy completeness (they sum to the difference between the prediction at the input and at the baseline), which makes them suitable for explaining individual network diagnoses relative to a healthy reference link.

2.4 Digital twins

A digital twin is a virtual representation of a physical system that is kept synchronised with it and used to analyse and predict its behaviour [4]. The network research community has described network digital twins as a means to validate configuration changes and what-if scenarios before applying them to production networks [5]. NetSage uses this idea specifically as a safety gate for automatic recovery actions.

2.5 Research gap

Existing approaches typically address only one part of the problem: diagnosis, prediction, explanation or validation. NetSage combines all four in a single closed loop, so that an automatic repair is predicted, explained and proven safe before it is executed, and is verified and rolled back if it does not work.

3. REQUIREMENTS

3.1 Functional requirements

1. Collect twelve telemetry features every second (simulation) or every 1.5 seconds (live).
2. Diagnose the network state into one of eight classes with a confidence score.
3. Estimate the risk of a fault within the next fifteen samples and forecast its type.
4. Explain each diagnosis with feature attributions, class probabilities and evidence sentences.
5. Maintain a digital twin and evaluate every candidate recovery action on it.
6. Execute only actions rated SAFE, verify the result, roll back or escalate on failure.
7. Support automatic mode and operator-approval mode, and predictive (preventive) healing.
8. Inject seven simulated faults and six live faults; undo live faults automatically.
9. Display all of the above on a web dashboard and expose it through a REST API.

3.2 Non-functional requirements

- **Safety:** no action may violate a safety constraint; live faults affect only the host computer.
- **Transparency:** every diagnosis and every accepted or rejected action has a visible reason.
- **Responsiveness:** detection within a few seconds of a fault; dashboard refresh every second.
- **Portability:** minimal dependencies (Flask, NumPy, psutil); runs on any machine with Python 3.9 or newer.

3.3 Hardware and software

Item	Requirement
Processor / memory	Any modern dual-core CPU, 4 GB RAM
Network adapter	Wi-Fi adapter (live mode only)
Operating system	Windows 10/11 for live mode; any OS for simulation mode
Language and libraries	Python 3.9+, NumPy, Flask, psutil
Front end	HTML5, CSS3, JavaScript (no external libraries), any modern browser
Windows tools used	netsh (Wi-Fi and firewall), ping, ipconfig, Native Wi-Fi API (wlanapi.dll)

Table 1 - Software and hardware requirements

4. SYSTEM ARCHITECTURE

NetSage is organised as a pipeline of five layers that run once per sample inside a single background engine. Figure 1 shows the layers and the data flowing between them.

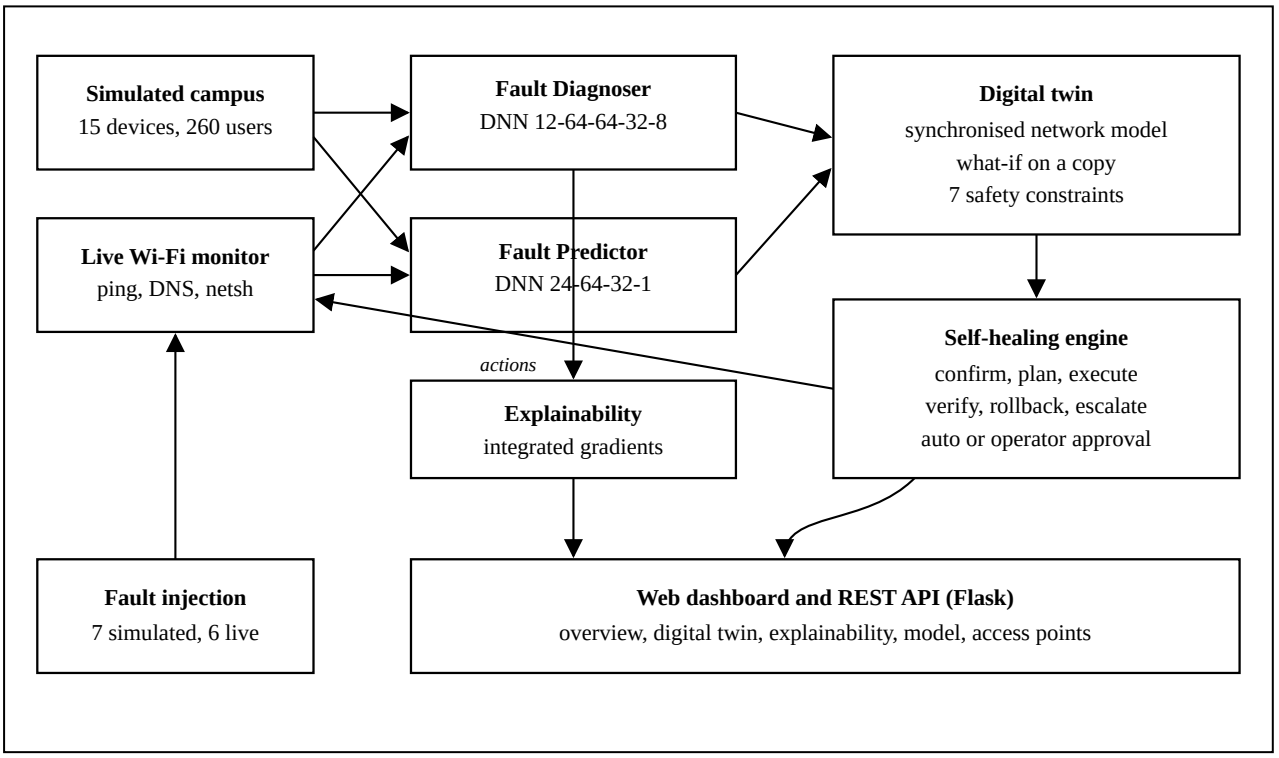


Figure 1 - System architecture

Data layer. Either the simulated enterprise network or the live Wi-Fi monitor produces the same twelve-feature vector, so the same trained models serve both modes.

Intelligence layer. The diagnoser classifies the current state, the predictor estimates near-term risk, and the explainability module attributes each decision to its inputs.

Twin layer. The digital twin mirrors the network state, projects trends into the future, and runs every candidate action on a copy of itself.

Control layer. The self-healing engine turns diagnoses and predictions into incidents, chooses the safest action, executes it, verifies the outcome and rolls back or escalates.

Presentation layer. A Flask server exposes a REST API and a single-page dashboard that refreshes every second.

5. MODULE DESIGN

5.1 Telemetry and features

Each sample contains the twelve features in Table 2. Unanswered probes are recorded as a 1000 ms timeout and 100% loss. Before entering the networks, latencies are log-scaled, the latency ratio is log-scaled up to 20x, and percentages are divided by 100, giving a fixed, scale-free normalisation that does not depend on the training data.

#	**Feature**	**Unit**	**Meaning**
1	Gateway latency	ms	Round-trip time to the default gateway
2	Gateway packet loss	%	Share of gateway probes lost
3	Internet latency	ms	Round-trip time to an internet host
4	Internet packet loss	%	Share of internet probes lost
5	Jitter	ms	Variation of internet latency
6	Latency vs baseline	x	Internet latency divided by the link's healthy baseline
7	DNS failure rate	%	Share of real DNS queries that failed
8	Wi-Fi signal	%	Received signal quality
9	Link state	0/1	Whether the access link is associated
10	Link utilisation	%	Load on the busiest link of the path
11	Interface errors	%	Physical-layer error rate
12	Device CPU	%	CPU load of the busiest device on the path

Table 2 - Telemetry features

Class	**Description**
Healthy	All paths and services respond normally
Link down	The access link is down; nothing is reachable
Packet loss	A significant share of packets is dropped
Congestion	A link is saturated and queues add heavy delay
Upstream outage	The gateway answers but the internet does not
DNS failure	IP connectivity works but name resolution fails
Wireless degraded	Poor radio conditions cause loss and delay
Device overload	A network device is CPU-bound and forwards slowly

Table 3 - Fault classes

5.2 Dataset generation

Labelled fault data from real enterprise networks is not publicly available, so a synthetic dataset was generated from documented fault signatures. Each sample starts from a randomised healthy link (internet baseline between 3 ms and 120 ms, covering a campus LAN up to a mobile hotspot) and applies a class-specific signature, for example high utilisation and a 2.2-15x latency rise for congestion, or a working gateway with 90-100% internet loss for an upstream outage. Overlapping ranges between classes and multiplicative and additive measurement noise were added deliberately so the task is not trivially separable. The diagnoser uses 3,000 samples per class (24,000 in total, split 80/20 into training and test sets). The predictor uses 2,500 synthetic time series containing healthy periods, gradual fault build-ups (10-30 samples), abrupt failures and short benign spikes, cut into 100,000 sliding windows.

5.3 Deep learning models

Both networks are fully connected deep neural networks implemented from first principles in NumPy: He weight initialisation, ReLU hidden activations, softmax or sigmoid output, cross-entropy loss, mini-batch Adam optimisation with cosine learning-rate decay and L2 regularisation. Implementing them directly keeps every step, including the input gradients used for explanations, transparent and dependency-free.

Property	**Fault Diagnoser**	**Fault Predictor**
Input	12 features (median of last 3 samples)	24 values: 12 recent levels + 12 trends over 10 samples
Layers	12 - 64 - 64 - 32 - 8	24 - 64 - 32 - 1
Activations	ReLU, softmax output	ReLU, sigmoid output
Parameters	7,336	3,713
Training / test samples	19,200 / 4,800	80,000 / 20,000 windows
Optimiser	Adam, cosine learning-rate decay, L2 = 1e-4, batch 128	
Epochs	25	20 (class-balanced weights)
Output	Probability of each of 8 classes	Probability of a fault within 15 samples

Table 4 - Neural network configuration

5.4 Explainability

Each diagnosis is explained in three complementary ways:

1. **Integrated gradients.** The input is moved from a healthy reference link to the current measurement in 32 steps; the gradient of the predicted class probability is accumulated along

this path and multiplied by the input difference. The result is the signed contribution of each feature, displayed as a ranked bar chart with percentage shares.

2. **Plain-language evidence.** Class-specific sentences quote the actual measurements, for example "The gateway still answers (8 ms) but 100% of internet probes are lost, so the fault lies beyond the gateway."
3. **Action reasons.** For every recovery action the digital twin lists the constraints it would violate or the reason it would not fix the fault.

5.5 Fault prediction

The predictor reads the last ten samples. When its risk output stays at or above 70% for three consecutive samples while the network is still healthy, an early warning is raised. To identify which fault is approaching, NetSage extrapolates every feature fifteen samples ahead using the Theil-Sen estimator [6][7] (the median of all pairwise slopes, which is robust to single noisy samples) and classifies the extrapolated state with the diagnoser.

5.6 Digital twin

The digital twin is an analytical model of the enterprise campus: three user groups (260 users) on three access points, two access switches, two redundant core switches, a data-centre switch with two DNS servers and an ERP application, an edge router and two internet providers (1 Gb/s primary, 500 Mb/s backup). From the current state it computes:

- **Routing** with Dijkstra's shortest path on configured link costs, including static default routes, provider load-sharing and QoS policing of bulk traffic.
- **Link load and delay:** utilisation u = load / capacity and queueing delay 1.5·u / (1 - u) ms (capped), with overflow loss above 100%.
- **Device CPU** from base load, forwarded traffic and control-plane storms, with forwarding delay and loss above 85% and 93% CPU.
- **Wi-Fi** signal, airtime load (clients / capacity) and interference from external sources and from co-channel access points on the same floor.
- **DNS** resolution of internal (erp.corp) and external names per user group.
- **Redundancy** as the number of edge-disjoint paths from each access switch to the edge (unit max-flow).

The twin is re-synchronised with the network every second and reports a **fidelity** score (agreement between its predicted and the measured latency), which stayed around 95-96% in our runs. For early warnings it keeps a short history of fault drivers (backup traffic, CPU storm level, interference level, link error rate) and projects them fifteen samples ahead, so preventive actions are tested against the predicted future network.

5.7 Recovery playbooks and safety constraints

For each fault class a playbook proposes three candidate actions, deliberately including tempting but dangerous options. Each candidate is applied to a copy of the twin and classified as **RECOMMENDED** (the best safe action), **SAFE**, **UNSAFE** (fixes the symptom but violates a constraint), **INEFFECTIVE** (does not fix the fault) or **BLOCKED** (precondition fails, for example the device is unreachable). Safe actions are ranked by disruption (user-minutes), worst link utilisation and known risks. The seven safety constraints are:

1. Every user group keeps internet, ERP and DNS (internal and external) reachability.
2. Packet loss ≤ 2% and latency ≤ 100 ms for all groups.
3. No link above 85% utilisation.
4. No device CPU above 85%.
5. No access point over its client capacity or on a clashing channel.
6. No access switch loses its redundant uplink.
7. Disruption within a budget of 60 user-minutes.

Injected fault	Type	Diagnosis	Chosen by the twin	Rejected by the twin (reason)
Cloud backup saturates ISP-A	Gradual	Congestion	QoS: police bulk traffic to 250 Mb/s	Move all traffic to ISP-B (ISP-B overloaded, 38% loss)
Broadcast storm on CORE-1	Gradual	Device overload	Control-plane policing and storm control	Shut CORE-1 (both access switches lose redundancy)
Fibre errors CORE-1 to ACC-1	Gradual	Packet loss	Drain traffic to the redundant link	Shut link (single point of failure); bounce (errors persist)
Radio interference on AP-1	Gradual	Packet loss / Wireless degraded	Move AP-1 to clean channel 157	Channel 149 (clashes with AP-2); maximum TX power (loss remains)
ISP-A provider outage	Abrupt	Upstream outage	Fail over to ISP-B with QoS (82% load)	Plain failover (ISP-B at 99%, 289 ms); reboot edge router (ineffective)
DNS-1 service crash	Abrupt	DNS failure	Fail clients over to DNS-2	Public DNS (internal names fail); restart (195 user-min disruption)
AP-1 radio failure	Abrupt	Link down	Split clients across AP-2 and AP-3	All clients to AP-2 (over capacity); remote reboot (AP unreachable)

Table 5 - Simulated faults and recovery decisions of the digital twin

5.8 Self-healing loop

The self-healing engine implements the control loop in Figure 2. A fault is confirmed only after the same diagnosis appears in three consecutive samples, which suppresses transient

misclassifications. If the diagnosed playbook contains no safe action, the twin searches the remaining playbooks in order of diagnosis probability; this allowed radio interference that first appeared as packet loss to be repaired with a channel change. After execution the engine waits for three consecutive healthy samples within the verification window (6 samples in simulation, 14 in live mode); otherwise the action is rolled back and the next safe candidate is tried. When no safe action remains, the incident is escalated to the operator with the full analysis. With Auto-heal disabled, the engine stops at the recommendation and waits for operator approval.

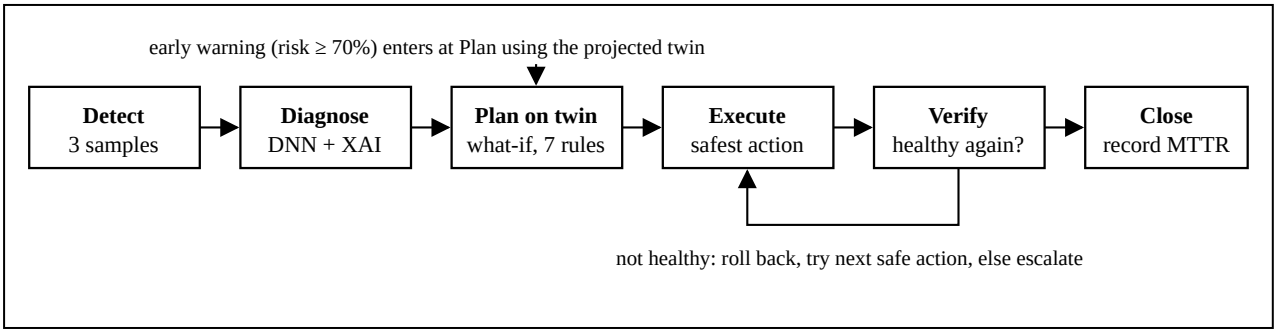


Figure 2 - Self-healing control loop

5.9 Fault injection

Simulation mode provides the seven faults of Table 5. Four build up gradually over 12-25 samples so that the predictor can observe them; three are abrupt and can only be detected. **Live mode** creates real faults on the host computer only, using standard Windows mechanisms. Every live fault has a timer, is removed on "Clear", on program exit, on console close and at the next start-up, and an emergency script (Fix-Network.bat) restores the network if the program is terminated abnormally.

Live fault	**Mechanism (this computer only)**	**Expected diagnosis**	**Recovery action chosen by the live twin**
Packet loss (40%)	Outbound firewall rule enabled and disabled at random	Packet loss	Remove the blocking rule found by the host policy audit
Internet outage	Firewall rule blocking public IP ranges	Upstream outage	Remove the blocking rule
DNS failure	Firewall rules blocking port 53	DNS failure	Remove the blocking rule
Wi-Fi disconnect	netsh wlan disconnect, held down for the duration	Link down	Reconnect to the saved profile
Adapter disabled	netsh interface set ... admin=disabled	Link down	Re-enable the adapter (reconnect is rated ineffective)
Congestion	Four parallel downloads (capped at 150 MB)	Congestion	Stop the bulk transfer

Table 6 - Live faults and recovery actions

The live twin models the path *host policy - Wi-Fi adapter - access point - gateway - upstream - DNS*. Its safety rules are: stay on the same authorised SSID, never select an access point marked BAD, remove only firewall rules created by NetSage, interrupt the connection for at most 10 seconds, and never terminate processes or change system DNS settings. Live mode also keeps a per-access-point scorecard (signal, loss, latency, operator mark) and steers the computer away from an access point marked BAD using the Windows Native Wi-Fi API.

6. IMPLEMENTATION

6.1 Technology stack

The back end is written in Python (about 3,500 lines across 21 modules, plus about 400 lines of tests) with NumPy for all numerical work and Flask for the web server. The dashboard is a single page written in plain HTML, CSS and JavaScript (about 800 lines) with charts drawn on HTML canvas and the topology drawn in SVG, so it works without internet access. Windows integration uses netsh, ping, ipconfig and the wlanapi.dll Native Wi-Fi API through ctypes.

6.2 Project structure

```
app.py                Flask server and REST API
netsage/features.py   12 telemetry features, 8 fault classes, normalisation
netsage/nn.py         deep neural network (forward, backpropagation, Adam,
gradients)
netsage/dataset.py    synthetic training data (static samples and time series)
netsage/models.py     Diagnoser, Predictor, integrated gradients, evidence
netsage/netmodel.py   analytical enterprise network model
netsage/simulation.py simulated production network with fault dynamics and noise
netsage/twin.py       digital twin: synchronisation, projection, what-if
evaluation
netsage/playbooks.py  recovery actions per fault class
netsage/healer.py     closed-loop self-healing state machine
netsage/engine.py     runtime loop for simulation and live modes
netsage/live/         probes, fault injector, live twin, AP scorecard, Wi-Fi CLI
web/                  dashboard (templates and static files)
models/               trained network weights and metrics
tests/                unit, end-to-end and mocked live tests
```

6.3 Key algorithm: twin-verified recovery

```
for class c in [diagnosed class] + other classes by probability:
    candidates = playbook(c, twin_state)              # projected state for
early warnings
    for action in candidates:
        if not action.precondition(state): verdict = BLOCKED; continue
        copy = clone(twin_state); action.apply(copy); kpis = evaluate(copy)
        if not fixes_symptom(kpis):  verdict = INEFFECTIVE
        elif violations(kpis):       verdict = UNSAFE
        else:                        verdict = SAFE
    if any SAFE: rank by disruption, utilisation, risk; break
execute best SAFE action -> verify -> rollback and try next -> escalate
```

6.4 REST API

Endpoint	Purpose
GET /api/state	Full snapshot: diagnosis, prediction, incidents, telemetry history, twin view, events
GET /api/model	Model architecture, training metrics and confusion matrix
POST /api/mode	Switch between simulation and live mode
POST /api/inject, /api/clear	Inject a fault; reset the network or clear the live fault
POST /api/settings	Turn Auto-heal and predictive healing on or off
POST /api/approve	Approve the recommended action (manual mode)
GET /api/whatif?code=	Run any playbook on the current twin
POST /api/ap/mark, /api/ap/connect, /api/ap/rescan	Access point marks, manual connection, rescan (live)

Table 7 - REST API

7. RESULTS AND DISCUSSION

7.1 Model performance

Model	Metric	Value
Fault Diagnoser	Test accuracy (4,800 held-out samples)	99.56%
	Classes	8
Fault Predictor	Accuracy (20,000 held-out windows)	94.70%
	Precision	99.25%
	Recall	86.97%
Training	Time for both networks on a laptop CPU	about 60 seconds

Table 8 - Model performance

Class	Recall	Class	Recall
Healthy	100.0%	Upstream outage	100.0%
Link down	100.0%	DNS failure	100.0%
Packet loss	98.4%	Wireless degraded	99.2%
Congestion	100.0%	Device overload	99.0%

Table 9 - Per-class recall of the diagnoser

The remaining errors occur between classes with genuinely overlapping symptoms: packet loss versus wireless degradation (both lose packets) and device overload versus wireless degradation (both add delay). The predictor is tuned for high precision so that preventive actions are rarely triggered without cause; its recall of 87% reflects that abrupt failures are inherently unpredictable. The high accuracy partly reflects the controlled synthetic data; accuracy on real enterprise telemetry is expected to be lower and would require retraining (Section 10).

7.2 End-to-end self-healing

Each fault was injected after 15 healthy samples and the system was observed for 60 samples, with predictive healing switched on and off.

Fault	Predictive ON	Predictive OFF
Cloud backup saturates ISP-A	Predicted, prevented (QoS)	Recovered (QoS)
Broadcast storm on CORE-1	Predicted, prevented (CoPP)	Recovered (CoPP)
Fibre errors CORE-1 to ACC-1	Recovered (drain link)	Recovered (drain link)
Radio interference on AP-1	Recovered (channel change)	Recovered (channel change)
ISP-A provider outage	Recovered (failover + QoS)	Recovered (failover + QoS)
DNS-1 service crash	Recovered (DNS-2)	Recovered (DNS-2)
AP-1 radio failure	Recovered (split clients)	Recovered (split clients)
150 samples, no fault	No incidents (no false alarms)	

Table 10 - End-to-end self-healing results

All fourteen runs ended with a healthy network. Gradual faults that build up slowly (backup traffic, CPU storm) were prevented before the diagnosis turned faulty; faster build-ups were caught reactively. In the dashboard runs, faults were diagnosed about 1-2.5 seconds after injection and service was restored within a few seconds of detection (for example 3 seconds for an ISP outage). In every scenario the twin rejected at least one plausible but unsafe or ineffective action, which demonstrates the value of verifying actions before execution.

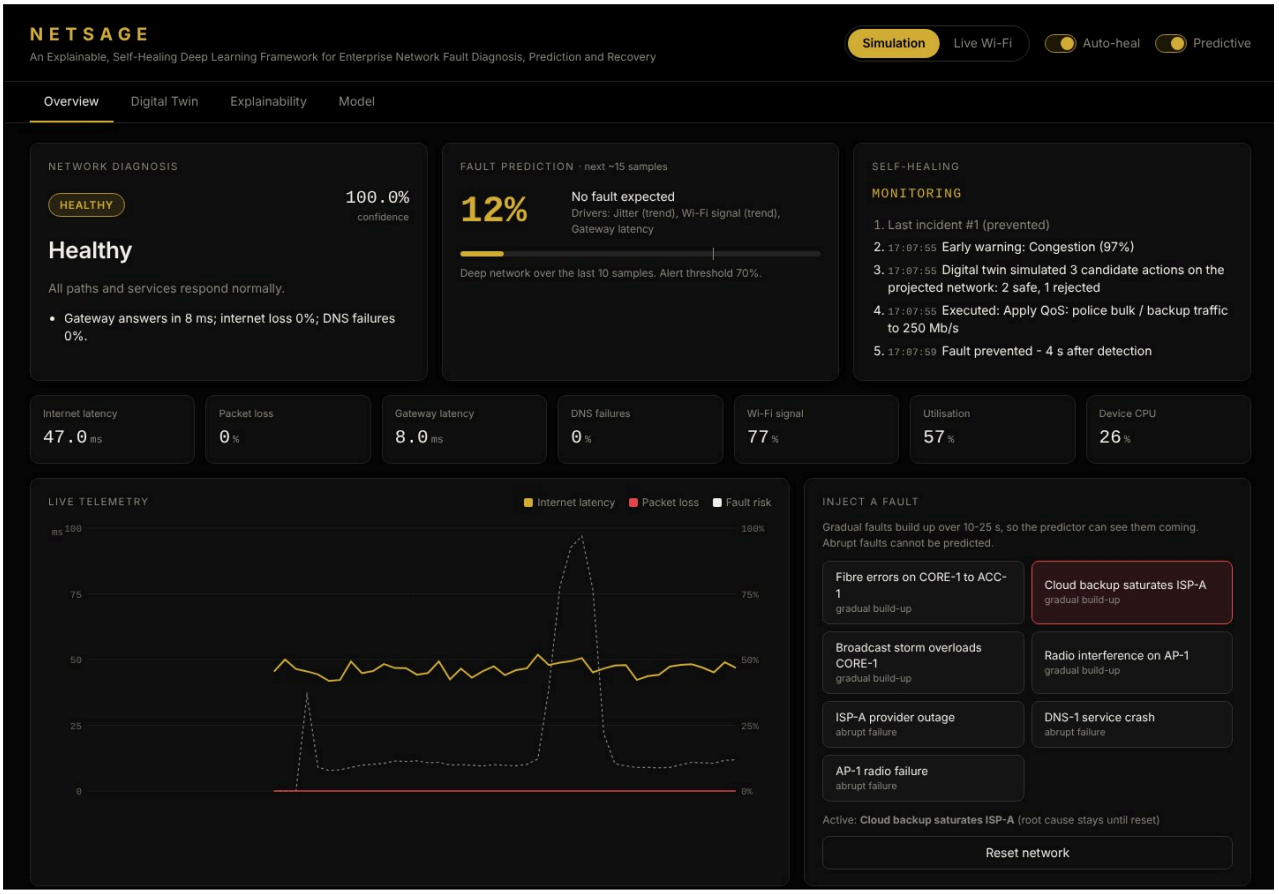


Figure 3 - Overview dashboard after a predicted congestion was prevented

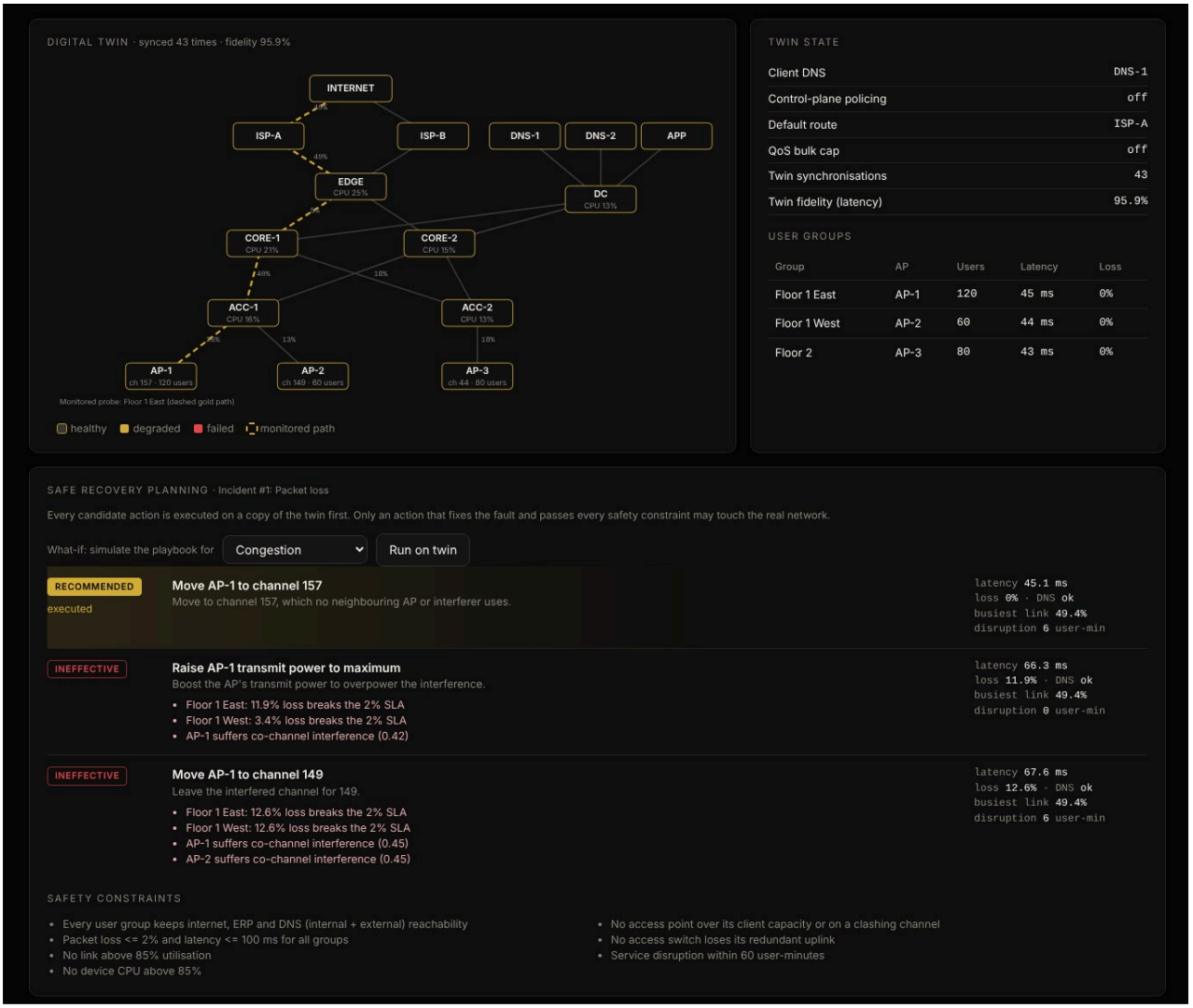


Figure 4 - Digital twin topology and safe recovery planning (radio interference incident)

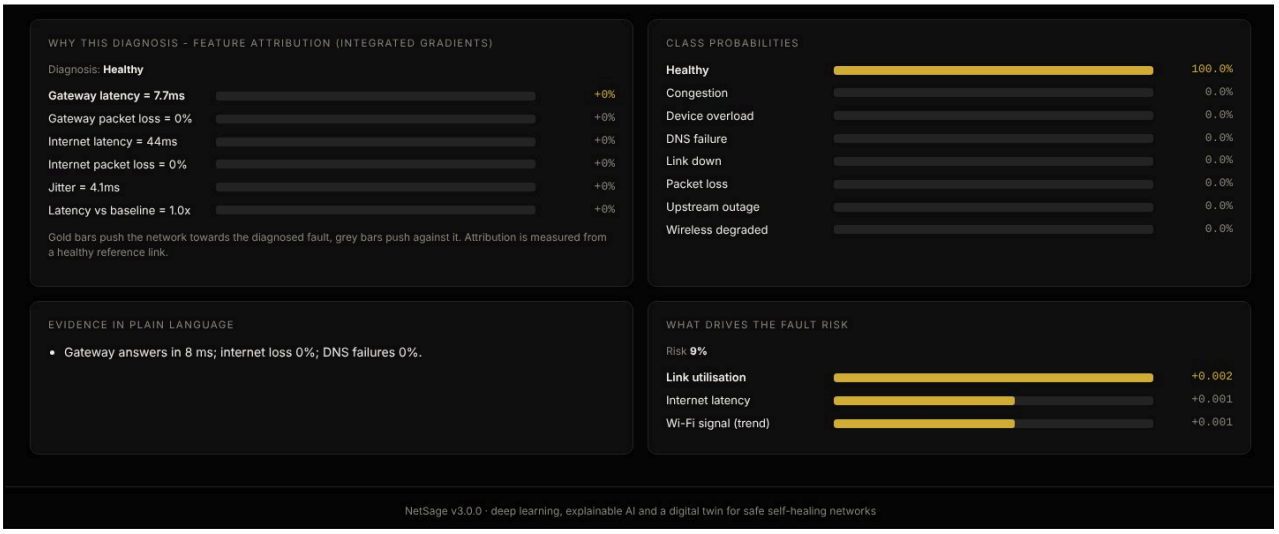


Figure 5 - Explainability view: feature attribution, class probabilities, evidence and risk drivers

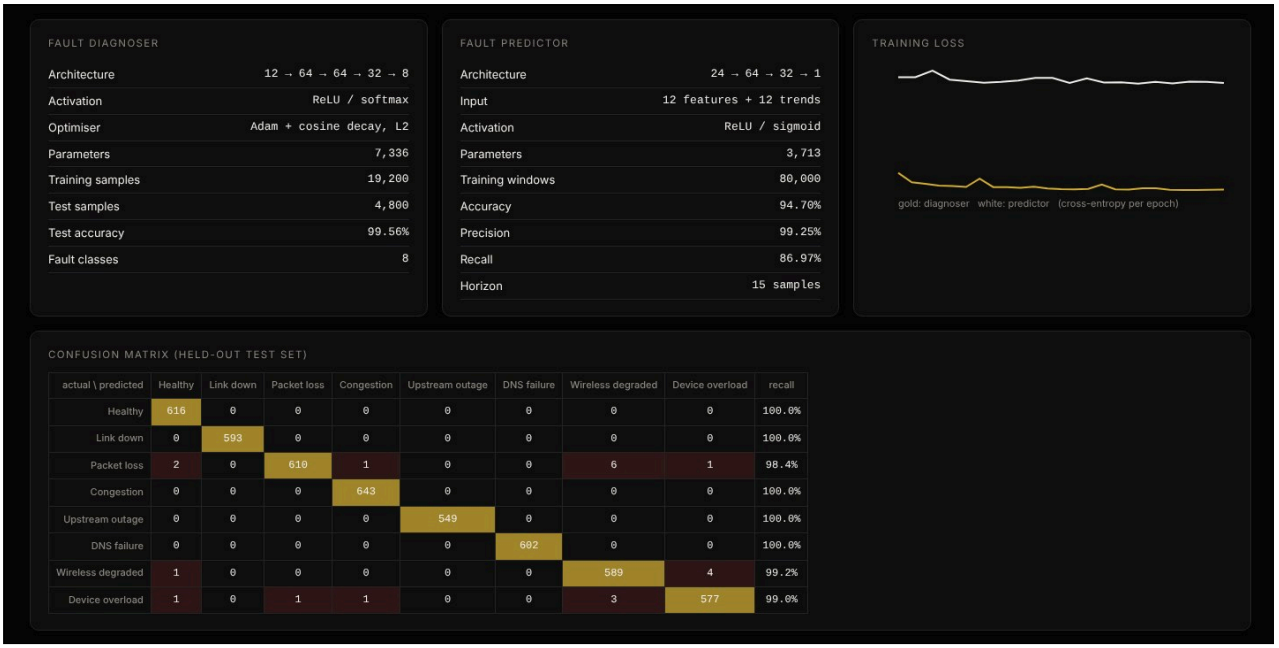


Figure 6 - Model performance view with confusion matrix

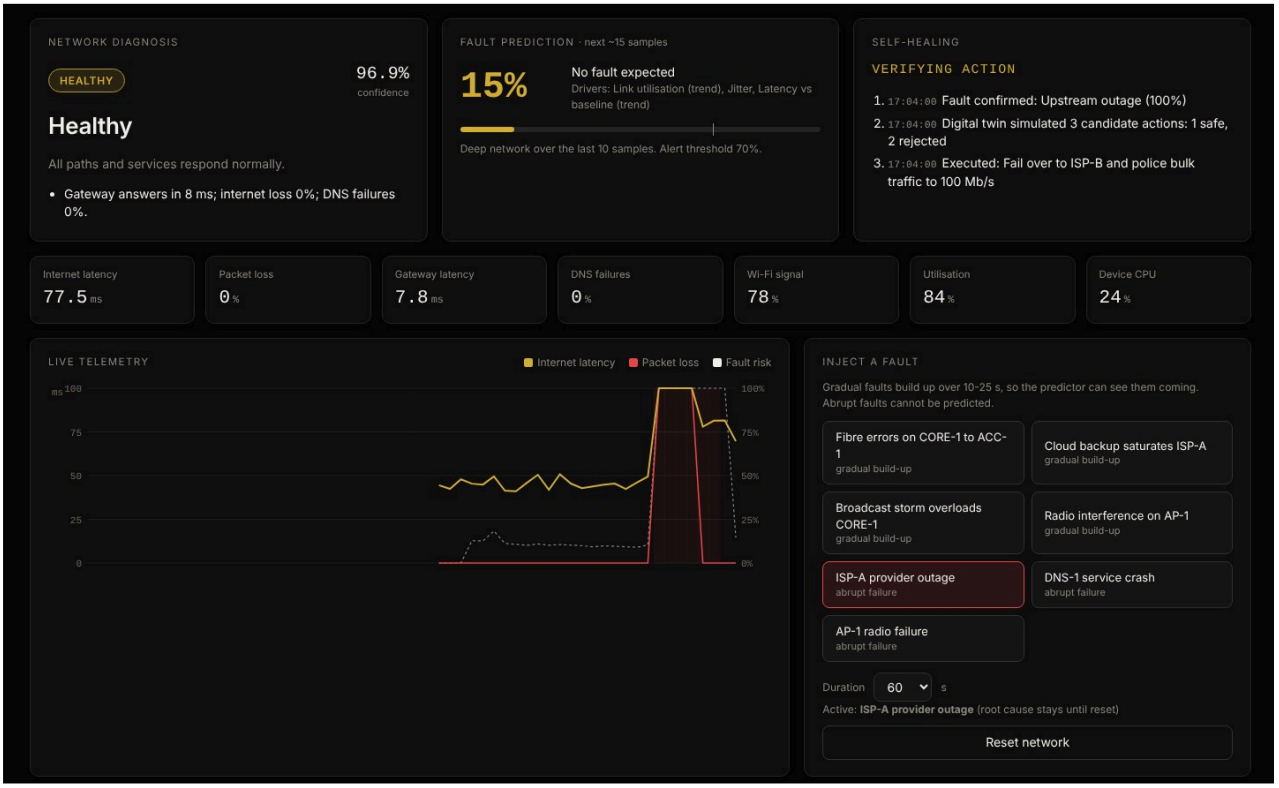


Figure 7 - Self-healing steps during an ISP outage

8. TESTING

An automated test suite of 16 tests (Python unittest) covers the neural network, the digital twin, the closed loop, the Windows parsers and the live mode. The live tests replace the Windows commands with a mocked network so the full live loop can be verified on any machine. All 16 tests pass.

#	Test case	Expected result	Status
1	Input gradient vs finite differences	Analytical gradients match numerical gradients	Pass
2	Diagnoser accuracy on freshly generated data	Accuracy > 95%	Pass
3	Explanation output	Six ranked factors and evidence sentences	Pass
4	Healthy baseline network	All groups reach internet and internal DNS	Pass
5	Twin decisions for all 7 faults	Expected action RECOMMENDED, at least one rejected	Pass
6	Closed loop, all 7 faults	Every incident recovered or prevented; network healthy	Pass
7	Manual approval mode	Waits for approval, then executes and verifies	Pass
8	No-fault run of 150 samples	Zero incidents	Pass
9-11	netsh interfaces, network scan and ping parsers	Correct BSSID, signal, channel, latency	Pass
12	Live: packet loss (mocked Windows)	Diagnosed, rule removed, recovered	Pass
13	Live: internet outage	Upstream outage, rule removed, recovered	Pass
14	Live: DNS failure	DNS failure, rule removed, recovered	Pass
15	Live: adapter disabled	Adapter re-enabled; reconnect rated not safe	Pass
16	Live: Wi-Fi disconnect	Reconnected, recovered	Pass

Table 11 - Test cases

9. USER MANUAL

9.1 Installation and start-up (Windows)

1. Install Python 3.9 or newer from python.org and tick "Add python.exe to PATH".
2. Extract NetSage.zip and open the folder containing Start-NetSage.bat.
3. Double-click Start-NetSage.bat and accept the administrator prompt. The first run creates a virtual environment and installs Flask, NumPy and psutil (about one minute).
4. The browser opens http://127.0.0.1:5000. Keep the console window open; closing it stops NetSage and undoes any live fault.
5. For live mode on Windows 11, turn on Settings > Privacy & security > Location and "Let desktop apps access your location", otherwise Windows hides Wi-Fi details.

On other operating systems: `pip install -r requirements.txt`, then `python app.py` (simulation mode). Tests: `python -m unittest discover -s tests -v`. Retraining: `python scripts/train.py`.

9.2 Using the dashboard

- **Overview:** diagnosis with confidence and evidence, fault risk, self-healing steps, KPI tiles, live chart, fault buttons, incident table and event log.
- **Digital Twin:** live topology with status and monitored path, twin state and policies, user groups, the evaluated actions of the current incident, the safety constraints, and a What-if tool to run any playbook on the current network.
- **Explainability:** feature attributions, class probabilities, evidence and risk drivers.
- **Model:** architecture, metrics, training loss and confusion matrix.
- **Access Points (live):** per-BSSID signal, loss, latency and quality; mark Good or Bad; connect manually.
- **Controls:** Simulation / Live Wi-Fi switch, Auto-heal (off = operator approval), Predictive (preventive healing).

9.3 Recommended demonstration

1. Turn Predictive off and inject "ISP-A provider outage"; observe diagnosis, twin planning and recovery.
2. Open the Digital Twin tab and explain the accepted and rejected actions; run a What-if.
3. Open the Explainability tab to show why the diagnosis was made.
4. Reset, turn Predictive on and inject "Cloud backup saturates ISP-A"; the incident is marked Prevented.
5. Turn Auto-heal off and inject "DNS-1 service crash"; approve the recommended action.
6. Switch to Live Wi-Fi, wait about 10 seconds, and inject a DNS failure on the computer.

10. LIMITATIONS

- The models are trained on synthetic data. Real enterprise telemetry is noisier and more varied, so accuracy in production would be lower until the models are retrained on real labelled data.
- The enterprise campus is an analytical model rather than a physical network; it was used to demonstrate the full closed loop safely and repeatably.
- Live mode runs on Windows and acts only on the host computer. Live faults are created by NetSage itself so that they can be demonstrated and undone, and the live twin only removes rules that NetSage created.
- Forcing a specific access point uses the Windows Native Wi-Fi API; the wireless driver may still roam on its own later.
- Abrupt failures (crashes, outages) cannot be predicted, only detected quickly.

11. CONCLUSION AND FUTURE WORK

11.1 Conclusion

NetSage demonstrates that automatic network recovery can be made both intelligent and trustworthy. Two deep neural networks diagnose faults with 99.56% test accuracy and predict gradual faults before they affect users. Integrated gradients and evidence sentences make each decision transparent. The digital twin acts as a safety gate: in every tested scenario it rejected at least one plausible action that would have overloaded a link, broken internal name resolution, removed redundancy, overloaded an access point or caused excessive disruption, and selected an action that restored service. Combined with confirmation, verification, rollback, escalation and optional operator approval, this closes the loop from detection to safe recovery, in both a simulated enterprise network and on a live Wi-Fi connection.

11.2 Future work

- Retrain and evaluate the models on real enterprise telemetry and public network datasets.
- Use sequence models (LSTM or Transformer) for longer-range fault prediction.
- Connect the twin and the actions to real devices through SNMP, streaming telemetry, NETCONF/RESTCONF or SDN controllers.
- Learn twin parameters automatically from measurements to raise fidelity.
- Use reinforcement learning, constrained by the twin's safety rules, to discover new recovery policies.
- Support multi-site networks and concurrent faults.

REFERENCES

1. D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *International Conference on Learning Representations (ICLR)*, 2015.
2. K. He, X. Zhang, S. Ren and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification," *IEEE International Conference on Computer Vision (ICCV)*, 2015.
3. M. Sundararajan, A. Taly and Q. Yan, "Axiomatic Attribution for Deep Networks," *International Conference on Machine Learning (ICML)*, 2017.
4. M. Grieves and J. Vickers, "Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems," in *Transdisciplinary Perspectives on Complex Systems*, Springer, 2017.
5. C. Zhou et al., "Network Digital Twin: Concepts and Reference Architecture," IRTF Network Management Research Group, Internet-Draft draft-irtf-nmrg-network-digital-twin-arch.
6. H. Theil, "A Rank-Invariant Method of Linear and Polynomial Regression Analysis," *Proceedings of the Royal Netherlands Academy of Sciences*, 1950.
7. P. K. Sen, "Estimates of the Regression Coefficient Based on Kendall's Tau," *Journal of the American Statistical Association*, vol. 63, 1968.
8. E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, 1959.
9. P. Mockapetris, "Domain Names - Implementation and Specification," RFC 1035, IETF, 1987.
10. Microsoft, "Native Wifi API (wlanapi.h) - WlanConnect function," Windows developer documentation.